License: CC BY 4.0 (https://info.arxiv.org/help/license/index.html#licenses-available)
arXiv:2610.14749v1 [cs.LO] 10 Oct 2026

M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.) \submissionTypeResearch Paper Track \correspondingauthor

Solving Distributed Flexible Job Shop Scheduling Problem Using SAT

†

Phuong Dang Anh

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam
24020277@vnu.edu.vn (mailto:24020277@vnu.edu.vn)

Duc Nguyen Kim Trung

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam
23021533@vnu.edu.vn (mailto:23021533@vnu.edu.vn)

To Van Khanh

Faculty of Information Technology, VNU University of Engineering and Technology, Hanoi, Vietnam
khanhtv@vnu.edu.vn (mailto:khanhtv@vnu.edu.vn)

2027

Abstract.

The Distributed Flexible Job Shop Scheduling Problem (DFJSP) requires the joint determination of factory assignment, machine selection, and operation sequencing to minimize the makespan. This paper presents an exact SAT-based approach built on an efficient SAT formulation that exploits problem structure through

propagation, symmetry breaking, and incremental SAT solving to effectively address these coupled decisions. The proposed method is evaluated on 76 standard benchmark instances from the Fattahi, Brandimarte, and *rdata* datasets under two- and three-factory configurations. Together with previously established results, the new SAT optimality certificates bring the total number of proven-optimal instances to 72 out of 76, including three newly certified instances; for two of these instances, SAT also improves the previous best-known makespans. Across the benchmark configurations, SAT achieves the best results on four of the five configurations and remains competitive with strong exact approaches such as CP and LBBD1_CPIL, as well as existing metaheuristic methods. These results demonstrate that the proposed SAT framework is an effective exact approach for the DFJSP, combining strong solution quality with rigorous optimality certification and offering a complementary exact method for future DFJSP studies and evaluations on additional benchmark instances.

Keywords: : Distributed flexible job shop scheduling problem, exact approach, SAT solving

1. Introduction

Production scheduling determines how operations are assigned to manufacturing resources and ordered over time while satisfying technological and capacity constraints. As manufacturing systems become more flexible, these decisions become more

difficult because an operation may be processed by different eligible resources rather than by a single predefined machine (Dauzère-Pérès et al., 2024).

The Job Shop Scheduling Problem (JSSP) (Graham, 1966) represents the classical setting in which each operation must be processed on a predefined machine. The Flexible Job Shop Scheduling Problem (FJSP) (Brucker and Schlie, 1990) extends this setting by allowing each operation to be processed on one of several eligible machines. Solving the FJSP therefore requires machine selection to be considered together with operation sequencing, which increases the difficulty of the scheduling problem (Brandimarte, 1993; Fattahi et al., 2007; Hurink et al., 1994; Mastrolilli and Gambardella, 2000). The Distributed Flexible Job Shop Scheduling Problem (DFJSP), introduced by De Giovanni and Pezzella (2010), further extends the FJSP to a multi-factory setting. In the DFJSP, each job is assigned to a single factory, and all of its operations are processed within that factory. A feasible schedule must therefore determine the factory assigned to each job, select an eligible machine for each operation, and determine the processing order of operations competing for the same machine (De Giovanni and Pezzella, 2010; Meng et al., 2020).

A wide range of heuristic and metaheuristic methods has been proposed for the DFJSP, including genetic algorithms, chemical

reaction optimization, and hybrid metaheuristics (De Giovanni and Pezzella, 2010; Chang and Liu, 2017; Lu et al., 2018; Marzouki et al., 2018; Xie et al., 2023). These methods can produce high-quality schedules for relatively large instances and have improved existing benchmark solutions. However, they do not in general provide certificates of optimality or rigorous lower bounds. Consequently, when a good feasible solution is obtained, it may remain unknown whether a better schedule exists. Exact methods address this limitation by providing optimality guarantees. Meng et al. (2020) developed several Mixed-Integer Linear Programming (MILP) formulations and a Constraint Programming (CP) model for the DFJSP, while Xiong and Liu (2026) later proposed Logic-based Benders Decomposition approaches that separate factory assignment from scheduling decisions. These methods have substantially advanced the exact solution of DFJSP benchmark instances. Nevertheless, proving optimality remains difficult for some medium- and large-scale cases within practical computational limits (Meng et al., 2020; Xiong and Liu, 2026).

These observations motivate the investigation of alternative exact approaches with two complementary capabilities. First, an exact method should be able to determine whether a known solution is optimal and, when possible, provide a proof of optimality. Second, it should remain effective in searching for improved feasible solutions when the current best-known makespan has not yet been certified. A method combining these capabilities is

particularly valuable for benchmark-driven research because it can both enlarge the set of instances with proven optimal solutions and improve the best-known results for unresolved cases.

Boolean Satisfiability (SAT) provides an alternative exact framework for discrete optimization. Modern Conflict-Driven Clause Learning SAT solvers combine Boolean propagation, clause learning, conflict analysis, and adaptive search strategies to efficiently solve large combinatorial problems (Biere et al., 2009). SAT-based formulations have also been successfully investigated for scheduling and related optimization problems (Koshimura et al., 2010; Huang and Zhou, 2018; Coelho and Vanhoucke, 2025). In particular, previous studies on the classical JSSP have shown that SAT can effectively represent temporal and machine-conflict decisions while supporting makespan minimization through repeated feasibility tests (Koshimura et al., 2010; Huang and Zhou, 2018). The DFJSP, however, introduces additional decisions that are absent from the classical JSSP, most notably job-to-factory assignment and alternative machine selection. These decisions interact with operation timing and resource conflicts and introduce additional symmetry when factories are identical. To the best of our knowledge, a dedicated SAT-based exact formulation for the DFJSP has not previously been studied. This work addresses this gap by developing a SAT-based approach for the minimum-makespan DFJSP and evaluating its ability both to certify optimality and to improve previously reported benchmark solutions.

The main contributions of this work are as follows:

- We propose an exact SAT-based approach for the DFJSP that exploits problem structure through an efficient formulation, problem-specific propagation, symmetry breaking, and incremental SAT solving.
- We provide a systematic computational evaluation on 76 standard benchmark instances from the Fattahi, Brandimarte (Brandimarte, 1993), and *rdata* (Hurink et al., 1994) datasets. The results establish three new optimality certificates, two of which correspond to instances for which SAT improves the previous best-known makespans, and, together with previously established results, show that 72 of the 76 instances have now been proven optimal.
- We provide an additional exact approach for the DFJSP that offers competitive solution quality and rigorous optimality certification, serving as a reference for future studies and evaluations on additional benchmark instances.

The remainder of this paper is organized as follows. Section 2 defines the DFJSP and reviews related work. Section 3 presents the proposed SAT formulation and the incremental optimization procedure. Section 4 describes the benchmark instances, experimental setup, and computational results. Finally, Section 5 concludes the paper and outlines directions for future research.

2. Problem Definition and Related Work

2.1. Problem Definition

The Distributed Flexible Job Shop Scheduling Problem (DFJSP) consists of a set of independent jobs $J = \{1, 2, ..., n\}$ to be processed in a set of factories $F = \{1, 2, ..., f\}$. All factories are assumed to be identical, meaning that they have the same machine configuration and that the eligibility and processing time of an operation on a given machine do not depend on the factory. Each factory contains the same set of machines $M = \{1, 2, ..., m\}$, and its production environment can therefore be viewed as a flexible job shop.

Each job $j \in J$ consists of an ordered sequence of n_j operations, $O_j = \{O_{j,1}, O_{j,2}, ..., O_{j,n_j}\}$, which must be processed according to a predefined technological order. Operation $O_{j,r}$ must therefore be completed before operation $O_{j,r+1}$ can start. Each operation $O_{j,r}$ can be processed on one machine selected from a predefined set of eligible machines $M_{j,r} \subseteq M$. The processing time of operation $O_{j,r}$ on an eligible machine $k \in M_{j,r}$ is denoted by $p_{j,r,k}$.

Each job is assigned to exactly one factory, and all operations

belonging to that job must be processed within the assigned factory. Within each factory, every operation is assigned to exactly one eligible machine, and operations assigned to the same machine must be processed without overlap.

The DFJSP considered in this work is defined under deterministic scheduling assumptions. All jobs, factories, and machines are available at time zero. Each machine can process at most one operation at a time, and operation processing is non-preemptive. Processing times are deterministic and known in advance. Machine breakdowns and setup times are not considered.

The objective is to minimize the makespan, defined as the completion time of the last finished job in the schedule $C_{\max} = \max_{j \in J} C_j$, where C_j denotes the completion time of job j. The optimal makespan is subsequently denoted by $C^*_{\max} = \min C_{\max}$.

Table 1 presents a small DFJSP instance consisting of three jobs, two identical factories, and three machines in each factory. Job J_1 and job J_2 each contain three operations, whereas job J_3 contains two operations. Each operation can be processed only on its eligible machines, with the corresponding processing times given in the table. Since the factories are identical, the eligibility and processing times shown in the table apply equally to every factory. Figure 1 illustrates the optimal schedule with

$C^*_{\max} = 8$.

Table 1. Illustrative DFJSP instance.

Job	Operation	Machine processing time		
		M_1	M_2	M_3
J_1	O_{11}	3	4	–
	O_{12}	–	2	3
	O_{13}	4	3	5
J_2	O_{21}	2	3	–
	O_{22}	4	–	3
	O_{23}	–	3	2
J_3	O_{31}	3	2	4
	O_{32}	4	3	–

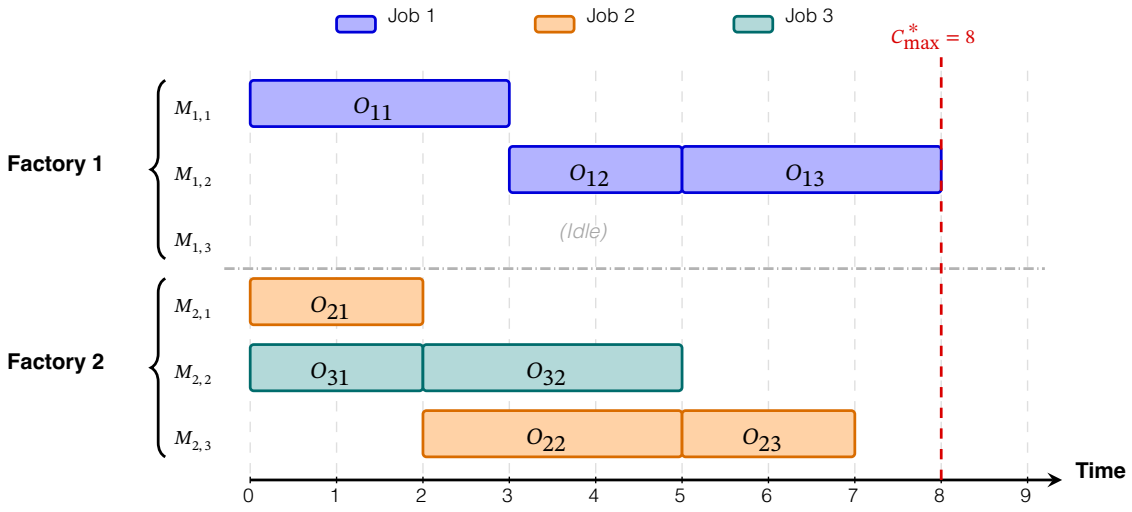


Figure 1. Optimal DFJSP schedule.

2.2. Related Work

Research on the DFJSP has mainly focused on two broad classes of solution approaches: heuristic and metaheuristic methods for efficiently obtaining feasible schedules, and exact methods aimed at providing optimality guarantees. More recently, advances in exact optimization have also led to decomposition-based approaches that exploit the hierarchical structure of factory assignment and shop-level scheduling.

Heuristic and metaheuristic approaches have been extensively investigated for the DFJSP because of the large search space arising from factory assignment, machine selection, and opera-

tion sequencing. Representative approaches include genetic algorithms, chemical reaction optimization, and other problem-specific search procedures (De Giovanni and Pezzella, 2010; Chang and Liu, 2017; Lu et al., 2018; Marzouki et al., 2018). More recently, Xie et al. (2023) proposed a Hybrid Genetic Tabu Search Algorithm (HGTSA) that combines genetic operators with critical-path-based tabu search and reported 13 new upper bounds on 69 benchmark instances. These approaches have played an important role in the search for competitive feasible schedules and in improving known benchmark upper bounds. However, heuristic and metaheuristic methods do not generally provide valid lower bounds or certificates of optimality.

Exact approaches complement heuristic search by providing mechanisms for establishing optimality. Meng et al. (2020) developed four Mixed-Integer Linear Programming (MILP) formulations and a Constraint Programming (CP) model for the DFJSP. Their computational results showed that the CP model scales more effectively than the MILP formulations on larger instances. The study established optimal solutions for 62 benchmark instances and reported 11 new best solutions, showing that exact methods can contribute both to optimality certification and to the improvement of known upper bounds. More recently, Xiong and Liu (2026) proposed Logic-based Benders Decomposition (LBBD) approaches that separate job-to-factory assignment from the scheduling subproblems associated with individual factories. Their best-performing variant, LBBD1_CPIL, com-

bines a master assignment problem with CP-based scheduling and reported an average optimality gap below 1.2% over 286 benchmark instances. These results substantially extended the range of DFJSP instances that can be addressed within an exact optimization framework, although proving optimality remains difficult for some medium- and large-scale instances.

Boolean Satisfiability (SAT) provides another exact paradigm for combinatorial scheduling and has previously been investigated for the classical Job Shop Scheduling Problem. Koshimura et al. (2010) formulated makespan minimization as a sequence of SAT feasibility problems using Boolean variables to represent temporal and machine-ordering decisions. Their approach proved optimal makespans for previously unresolved benchmark instances and improved several known bounds. Huang and Zhou (2018) subsequently developed a more compact SAT formulation by restricting temporal variables and clauses to feasible time ranges and introducing precedence variables only for operations that can compete for the same machine. These studies indicate that SAT can be effective for exact scheduling when the Boolean formulation exploits problem-specific structure to strengthen propagation and avoid unnecessary variables and clauses.

However, SAT formulations developed for the classical JSSP do not directly capture the additional decisions introduced by the

DFJSP. In the JSSP, each operation is assigned to a predefined machine and all operations are processed within a single shop. In the DFJSP, each job must additionally be assigned to a factory, while each operation must select one of several eligible machines. Consequently, machine-capacity conflicts are conditional on both factory and machine assignments: two operations compete for the same physical resource only when their jobs are assigned to the same factory and both operations select the same machine. Moreover, identical factories introduce symmetries associated with interchangeable factory labels. These characteristics require factory assignment, machine selection, temporal decisions, and conditional resource conflicts to be represented jointly within the Boolean formulation.

Existing exact approaches for the DFJSP have primarily relied on MILP, CP, and decomposition-based methods, while SAT-based scheduling studies have addressed various scheduling settings. The proposed SAT formulation jointly models factory assignment, machine selection, timing, precedence, and conditional machine-capacity constraints, supported by temporal preprocessing, propagation, symmetry breaking, and incremental SAT solving.

3. SAT Solving for the Distributed Flexible Job Shop Scheduling Problem

For the formulation, all operations are indexed globally by $i \in O$, and $J(i)$ denotes the job containing operation i. Let E denote the set of direct precedence relations, M_i the set of eligible machines for operation i, and $p_{i,k}$ its processing time on machine $k \in M_i$.

3.1. Upper-Bound Construction and Preprocessing

A finite makespan bound is required to construct the time-indexed SAT encoding because it determines the temporal range over which start-time variables are instantiated. A smaller valid upper bound therefore reduces the number of Boolean variables and clauses and provides the initial bound for the subsequent makespan optimization.

Before constructing the SAT model, we compute an initial feasible schedule to obtain an upper bound UB on the makespan. Let $p_i^{\min} = \min_{k \in M_i} p_{i,k}$ denote the minimum processing time of operation i. Since the operations of each job form a linear precedence chain, let $\mathrm{pred}(i)$ and $\mathrm{succ}(i)$ denote the immediate predecessor and successor of operation i, respectively, whenever they exist. The workload of job j is estimated as

$$W_j = \sum_{i : J(i) = j} p_i^{\min}. \tag{1}$$

Jobs are first sorted in non-increasing order of W_j and assigned sequentially to the factory with the smallest accumulated workload. Once the factory assignments are fixed, the operations are scheduled greedily. A tail length is computed for each operation according to

$$L_i = \begin{cases} p_i^{\min}, & \text{if } i \text{ has no successor,} \\ p_i^{\min} + L_{\mathrm{succ}(i)}, & \text{otherwise.} \end{cases} \tag{2}$$

Among the currently available operations, the operation with the largest tail length is selected. For each eligible machine k in the assigned factory, the operation is scheduled at the maximum of its precedence-based ready time and the current availability time of machine k. The machine yielding the smallest resulting completion time is selected. The makespan of the resulting feasible schedule is used as UB.

Using UB, precedence-based temporal bounds are then computed for each operation. The earliest start times are obtained by forward propagation:

$$ES_i = \begin{cases} 0, & \text{if } i \text{ is the first operation of its job,} \\ ES_{\mathrm{pred}(i)} + p_{\mathrm{pred}(i)}^{\min}, & \text{otherwise.} \end{cases} \tag{3}$$

The latest start times are initialized from UB and tightened by backward propagation:

$$LS_i = \begin{cases} UB - p_i^{\min}, & \text{if } i \text{ is the last operation of its job,} \\ LS_{\mathrm{succ}(i)} - p_i^{\min}, & \text{otherwise.} \end{cases} \tag{4}$$

The resulting interval $[ES_i, LS_i]$ defines the temporal domain of operation i and is used to restrict the time-indexed variables in the SAT formulation.

3.2. SAT Formulation

Using the temporal domains obtained above, we construct the SAT formulation. The eligible machines of each operation are ordered by non-decreasing processing time. Let $M_i^{'} = (k_{i,1}, k_{i,2}, \ldots, k_{i,|M_i|})$ denote this ordering, such that $p_{i,k_{i,1}} \leq p_{i,k_{i,2}} \leq \cdots \leq p_{i,k_{i,|M_i|}}$.

Boolean variables.

The Boolean variables used in the formulation are summarized in Table 2.

Table 2. Boolean variables used in the SAT formulation.

Variable	Meaning
$f_{j,q}$	True iff job j is assigned to factory q.
$sf_{j,\ell}$	True iff jobs j and ℓ are assigned to the same factory.
$m_{i,k}$	True iff operation i is processed on machine k.
$xm_{i,r}$	True iff the machine selected for operation i is at position r or later in the processing-time-ordered list M_i'.
$s_{i,t}$	True iff operation i starts at time t.
$x_{i,t}$	True iff operation i starts at or after time t.
$sm_{i,h,k}$	True iff operations i and h both select machine k.
$sfm_{i,h,k}$	True iff operations i and h use machine k in the same factory.

Factory assignment constraints.

Since all factories are identical, permuting their indices does not change the feasibility or makespan of a schedule. We therefore restrict the factories available to job j to $F_j = \{1, ..., \min(j, f)\}$. Thus, the first job can only be assigned to factory 1, the second job can use factories 1 and 2, and so on. Under this restricted domain, each job is assigned to exactly one factory:

$$\sum_{q \in F_j} f_{j,q} = 1, \qquad \forall\, j \in J. \tag{5}$$

Machine assignment constraints.

The machine-selection variables are encoded using the processing-time ordering of $M_i^{'}$. Specifically, the order variables form a monotone sequence, and the selected machine is identified by the unique transition from true to false in this sequence:

$$\begin{aligned} & xm_{i,1} = \text{True}, \\ & xm_{i,r+1} \rightarrow xm_{i,r}, && r = 1, ..., |M_i| - 1, \\ & m_{i,k_{i,r}} \leftrightarrow \left(xm_{i,r} \wedge \neg\, xm_{i,r+1} \right), && r = 1, ..., |M_i| - 1, \\ & m_{i,k_{i,|M_i|}} \leftrightarrow xm_{i,|M_i|}. \end{aligned} \tag{6}$$

Start-time encoding constraints.

Similarly to the machine-assignment encoding, the start time of each operation i is encoded over its temporal domain $[ES_i, LS_i]$ using order variables:

$$
\begin{aligned}
& x_{i,ES_i} = \text{True}, \\
& x_{i,t+1} \rightarrow x_{i,t}, && t = ES_i, \ldots, LS_i - 1, \\
& s_{i,t} \leftrightarrow \left(x_{i,t} \wedge \neg\, x_{i,t+1}\right), && t = ES_i, \ldots, LS_i - 1, \\
& s_{i,LS_i} \leftrightarrow x_{i,LS_i}.
\end{aligned} \tag{7}
$$

Direct precedence constraints.

For each precedence relation $(i,h) \in E$ and start time $t \in [ES_i, LS_i]$, we first establish a separation based on the minimum processing time $p_i^{\min}$:

$$
\begin{aligned}
& s_{i,t} \rightarrow x_{h,t+p_i^{\min}}, && ES_h < t + p_i^{\min} \leq LS_h, \\
& \neg\, s_{i,t}, && t + p_i^{\min} > LS_h.
\end{aligned} \tag{8}
$$

To account for slower machine alternatives, this separation is conditionally tightened for each position $r = 2, \ldots, |M_i|$ satisfying $p_{i,k_{i,r}} > p_{i,k_{i,r-1}}$:

$$
\begin{aligned}
& s_{i,t} \wedge xm_{i,r} \rightarrow x_{h,t+p_{i,k_{i,r}}}, && ES_h < t + p_{i,k_{i,r}} \leq LS_h, \\
& s_{i,t} \rightarrow \neg\, xm_{i,r}, && t + p_{i,k_{i,r}} > LS_h.
\end{aligned} \tag{9}
$$

In both cases, instances with $t + p \leq ES_h$ are trivially satisfied by the time window of h and omitted.

Transitive precedence constraints.

Let E^* denote the transitive closure of E. Because the operations of each job form a linear chain, there exists a unique precedence path $P(i,h)$ from operation i to any downstream operation h, with a cumulative minimum processing time $w_{i,h} = \sum_{v \in P(i,h) \setminus \{h\}} p_v^{\min}$. For each non-direct relation $(i,h) \in E^* \setminus E$ and each start time $t \in [ES_i, LS_i]$, we enforce:

$$\begin{aligned} & s_{i,t} \rightarrow x_{h,\,t+w_{i,h}}, && ES_h < t + w_{i,h} \leq LS_h, \\ & \neg\, s_{i,t}, && t + w_{i,h} > LS_h. \end{aligned} \tag{10}$$

Cases where $t + w_{i,h} \leq ES_h$ are inherently satisfied by the temporal domain of h and omitted. While these constraints do not alter the set of feasible schedules, they enable the temporal implications of composite precedence chains to be propagated directly via unit propagation.

Machine-capacity constraints.

Because each machine can process at most one operation at a time, machine-capacity constraints must ensure that operations sharing the same machine within the same factory do not overlap in time. These constraints are required only for unordered pairs of operations $\{i,h\} \subseteq O$ with $(i,h),(h,i) \notin E^*$ that share an eligible machine $k \in M_i \cap M_h$.

Let $j = \min\{J(i), J(h)\}$ and $\ell = \max\{J(i), J(h)\}$. Given that $F_j \subseteq F_\ell$

for $j < \ell$, the conditions that the two operations are assigned to the same factory, select the same machine, and consequently use the same physical resource are formulated as:

$$
\begin{aligned}
& f_{j,q} \wedge f_{\ell,q} \rightarrow sf_{j,\ell}, && q \in F_j, \\
& sf_{j,\ell} \wedge f_{j,q} \rightarrow f_{\ell,q}, && q \in F_j, \\
& sm_{i,h,k} \leftrightarrow \left(m_{i,k} \wedge m_{h,k}\right), \\
& sfm_{i,h,k} \leftrightarrow \left(sm_{i,h,k} \wedge sf_{j,\ell}\right).
\end{aligned}
\tag{11}
$$

The machine capacity limit dictates that operations i and h cannot be executed concurrently on machine k. We enforce this capacity constraint by requiring that if both operations are assigned to machine k and operation i starts at time t, operation h must either complete before i starts or start after i completes. For each $t \in [ES_i, LS_i]$, letting $A = t - p_{h,k}$ and $B = t + p_{i,k}$, this requirement is encoded as:

$$
sfm_{i,h,k} \wedge s_{i,t} \rightarrow (\neg\, x_{h,A+1} \vee x_{h,B}),
\tag{12}
$$

where literal $\neg\, x_{h,A+1}$ is included only when $A \geq ES_h$, and $x_{h,B}$ is included only when $B \leq LS_h$. Instances of Equation (12) with $A \geq LS_h$ or $B \leq ES_h$ are trivially satisfied, and thus no clause is generated. Conversely, when both $A < ES_h$ and $B > LS_h$, neither literal is included on the right-hand side, resulting in a 2-literal clause $\neg\, sfm_{i,h,k} \vee \neg\, s_{i,t}$ (i.e., $sfm_{i,h,k} \wedge s_{i,t} \rightarrow \bot$), which forbids operation i from starting at time t whenever the two operations

share machine k in the same factory.

Symmetry breaking.

The restricted factory domains F_j eliminate some permutations of identical factories but do not remove all equivalent assignments. To break the remaining symmetries, we impose the condition that factory q can be used only if factory $q-1$ has already been assigned to an earlier job:

$$f_{j,q} \rightarrow \bigvee_{h=q-1}^{j-1} f_{h,q-1}, \qquad j = 2, \ldots, |J|, \quad q \in F_j \setminus \{1\}. \tag{13}$$

3.3. Incremental SAT Solving

Because SAT is fundamentally a decision problem, SAT solvers cannot directly optimize an objective function. Therefore, makespan minimization must be achieved by solving a sequence of decision problems, each querying whether a feasible schedule exists within a candidate makespan bound U.

To enforce a specific candidate bound U in the SAT formulation, it is sufficient to restrict the completion times of the last operations of all jobs. Let O^{last} denote the set of these final operations. For each $i \in O^{\text{last}}$ and each eligible machine $k \in M_i$, we define the machine-dependent latest start time:

(14) $$\tau_{i,k}(U) = U - p_{i,k}.$$

If operation i is assigned to machine k, it must start no later than $\tau_{i,k}(U)$ to ensure the entire job completes within the current makespan bound. The corresponding constraints are:

(15) $$\begin{aligned} &\neg\, m_{i,k}, && \tau_{i,k}(U) < ES_i, \\ &m_{i,k} \rightarrow \neg\, x_{i,\tau_{i,k}(U)+1}, && ES_i \leq \tau_{i,k}(U) < LS_i. \end{aligned}$$

When $\tau_{i,k}(U) \geq LS_i$, no additional clause is required because the temporal domain of operation i already guarantees its completion within U when machine k is selected.

Algorithm 1 summarizes the incremental optimization procedure. The base SAT formulation is constructed once using the temporal domains derived from the initial upper bound UB. Starting from UB, makespan clauses corresponding to the current candidate bound are incrementally added to the same solver instance. Whenever a feasible solution is obtained, its actual makespan is used to tighten the next bound directly.

Algorithm 1 Incremental makespan optimization

SAT solver S containing the base formulation, initial upper bound UB
Optimal schedule and optimal makespan $C^*_{\max}$
$U \leftarrow UB$
$bestSchedule \leftarrow \emptyset$
$C^*_{\max} \leftarrow UB$
while true **do**
    Add the makespan constraints of Eq. (15) for bound U to S
    $(status, schedule) \leftarrow \textsc{Solve}(S)$
    if $status$ = UNSAT **then**
        return $(bestSchedule, C^*_{\max})$
    end if
    $C_{\max} \leftarrow \textsc{Makespan}(schedule)$
    $bestSchedule \leftarrow schedule$
    $C^*_{\max} \leftarrow C_{\max}$
    $U \leftarrow C_{\max} - 1$
end while

The same solver instance is retained throughout the optimization process, so all previously added clauses and learned information remain available in subsequent iterations. After obtaining a feasible schedule with makespan $C_{\max}$, the next bound is set directly to $C_{\max} - 1$. Because makespan constraints are added monotonically, all previously imposed upper bounds remain valid and are implied by the tighter bound. When the formulation becomes unsatisfiable, no feasible schedule exists with a makespan smaller than the last feasible $C_{\max}$. Therefore, the last feasible schedule is optimal and its makespan is $C^*_{\max}$.

4. Computational Experiments

4.1. Benchmark Instances

We evaluate the proposed SAT method on three widely used FJSP benchmark suites: Brandimarte (Brandimarte, 1993), Fattahi et al. (Fattahi et al., 2007), and the *rdata* subset of Hurink et al. (Hurink et al., 1994). Since these benchmarks were originally designed for single-factory production, we use the distributed versions adopted in previous DFJSP studies (Meng et al., 2020; Xie et al., 2023), in which identical factories share the same machine configurations and processing capabilities. The Fattahi set contains 10 instances and is evaluated with two factories, while the Brandimarte set contains 10 instances and is tested with both two and three factories. The *rdata* set contains 23 instances and is also evaluated with two and three factories. We omit the four-factory *rdata* setting because previous studies (Meng et al., 2020; Xie et al., 2023) show that almost all instances already attain their theoretical lower bounds with three factories, while the four-factory configuration becomes nearly trivial and therefore provides limited discriminatory value.

4.2. Experimental Setup and Metrics

All SAT experiments are conducted on a Google Cloud Compute Engine *c4-standard-4* virtual machine. The proposed encoding is implemented in Python using PySAT (Ignatiev et al., 2024),

with CaDiCaL version 1.9.5 (Biere et al., 2020) as the underlying solver. Each instance is evaluated under 600 s and 1800 s time limits, including preprocessing, encoding construction, and solving. The 1800 s run is omitted when the corresponding instance has already been proven optimal within 600 s.

The proposed method is compared with the MILP and CP models of Meng et al. (Meng et al., 2020), HGTSA (Xie et al., 2023), and LBBD1_CPIL (Xiong and Liu, 2026). For the CP model of Meng et al., two sets of results are reported separately, namely the original 600 s results from Meng et al. (Meng et al., 2020) and the 1800 s results obtained by Xiong et al. (Xiong and Liu, 2026) using the same model. All baseline results are taken from the corresponding publications or publicly released experimental data, whereas only the proposed SAT method is executed on the platform described above.

HGTSA (Xie et al., 2023) is run for up to 4000 iterations rather than under a fixed time limit. As a metaheuristic, it does not itself prove optimality; therefore, a reported HGTSA solution is regarded as optimal when its makespan matches the corresponding reported lower bound. To ensure consistency with previous literature, we consider only the 20 two-factory *rdata* instances reported by Xiong and Liu (2026) (for LBBD1_CPIL and CP) that match the benchmark set used in earlier studies. The aggregate metrics for LBBD1_CPIL and the CP rerun are therefore comput-

ed over these 20 common instances[1] [[1] The corresponding data and results are publicly available at https://github.com/FuliXiong/Instances-and-Results-for-DFJSP (https://github.com/FuliXiong/Instances-and-Results-for-DFJSP)] .

For reproducibility, both the source code of the proposed SAT method and the detailed per-instance experimental results are provided in the Supplementary Material accompanying this paper.

We use the following metrics to evaluate solution quality and computational effectiveness:

- *#Optimal*: The number of instances for which the reported makespan is established as optimal. For HGTSA, optimality is inferred when its reported best makespan matches the reported lower bound.
- *#Best*: The number of instances for which a method obtains the lowest makespan among all compared approaches. Ties are credited to all corresponding methods.
- *#New*: The number of instances for which SAT obtains a makespan strictly smaller than the previously best-known published value.

- *Avg. Dev. (%)*: The average percentage deviation from the best-found makespan over the evaluated instance set I:

$$\text{Avg. Dev. } (\%) = \frac{1}{|I|} \sum_{i \in I} \frac{C_{\max,i} - C^{\text{best}}_{\max,i}}{C^{\text{best}}_{\max,i}} \times 100\,\%\,.$$

  Here, $C_{\max,i}$ denotes the makespan obtained by the evaluated method for instance i, while $C^{\text{best}}_{\max,i}$ denotes the minimum makespan reported among all compared methods for that instance.

4.3. Results and Analysis

Table 3. Comparison of experimental results across different datasets and methods.

ries	#Instances	Metric	MILP (Meng et al., 2020) (600s)	CP (Meng et al., 2020) (600s)
	10	#Optimal	3	–
		#Best	3	–
		#New	–	–
		Avg. Dev. (%)	50.17	–

10	#Optimal	–	–
	#Best	–	–
	#New	–	–
	Avg. Dev. (%)	–	–
10	#Optimal	**10**	–
	#Best	**10**	–
	#New	–	–
	Avg. Dev. (%)	**0.00**	–
23	#Optimal	13	**17**
	#Best	13	17
	#New	–	–
	Avg. Dev. (%)	1.45	0.35
23	#Optimal	20	22
	#Best	20	22
	#New	–	–
	Avg. Dev. (%)	0.55	0.01

- "–" denotes not applicable, unreported results, or omitted runs where SAT (600s) already solved all instances to optimality.

* Values are computed over the 20 common two-factory *rdata* instances reported by Xiong et al. (Xiong and Liu, 2026).

Table 3 summarizes the computational results across the five benchmark configurations. Within 600 s, SAT obtains the best makespan among the compared methods on 68 of the 76 instances and proves optimality for 66 instances. Extending the unresolved runs to 1800 s increases these numbers to 69 and 67, respectively. SAT proves all Fattahi instances optimal, matching the complete result previously reported for MILP, and also solves all three-factory Brandimarte and *rdata* instances to proven optimality within 600 s. The remaining unresolved cases are concentrated in the two-factory Brandimarte and *rdata* configurations, where the smaller number of factories leads to stronger competition for machine resources and more restrictive scheduling decisions. Across the five benchmark configurations, SAT achieves or matches the strongest overall performance in four of them: two-factory Brandimarte, three-factory Brandimarte, two-factory Fattahi, and three-factory *rdata*. The exception is the two-factory *rdata* configuration, where HGTSA obtains the best makespan on all 23 instances, although SAT still proves 17 instances optimal and remains competitive in solution quality.

The two-factory configurations provide the most informative comparison among the exact approaches. On Brandimarte, SAT and the CP rerun of Xiong et al. (Xiong and Liu, 2026) both prove seven instances optimal within 1800 s. However, SAT obtains the best makespan on nine instances, compared with seven for CP, and achieves an average deviation of 0.11%, compared with 0.88% for CP. LBBD1_CPIL obtains the best makespan on eight

instances but proves four instances optimal. On the two-factory *rdata* configuration, the original CP results of Meng et al. (Meng et al., 2020) and SAT each prove 17 of the 23 instances optimal. HGTSA obtains the best reported makespan on all 23 instances, while SAT achieves 17 best solutions and an average deviation of 0.24% after 1800 s. LBBD1_CPIL proves 15 of the 20 reported instances optimal and obtains the best makespan on 16 of them. These results indicate that SAT is competitive not only in optimality certification but also in solution quality on the unresolved instances.

The difference between the two- and three-factory configurations is noticeable for Brandimarte and *rdata*. With three factories, SAT proves all 10 Brandimarte and all 23 *rdata* instances optimal within 600 s. For Brandimarte, the CP rerun reaches the same best makespan on all 10 instances but certifies only eight within 1800 s, while LBBD1_CPIL certifies five. For *rdata*, the previously reported results for CP and HGTSA attain the corresponding best values on 22 of the 23 instances, whereas SAT proves all 23 optimal. These results suggest that increasing the number of factories substantially reduces the difficulty associated with machine-resource conflicts, despite enlarging the factory-assignment decision space.

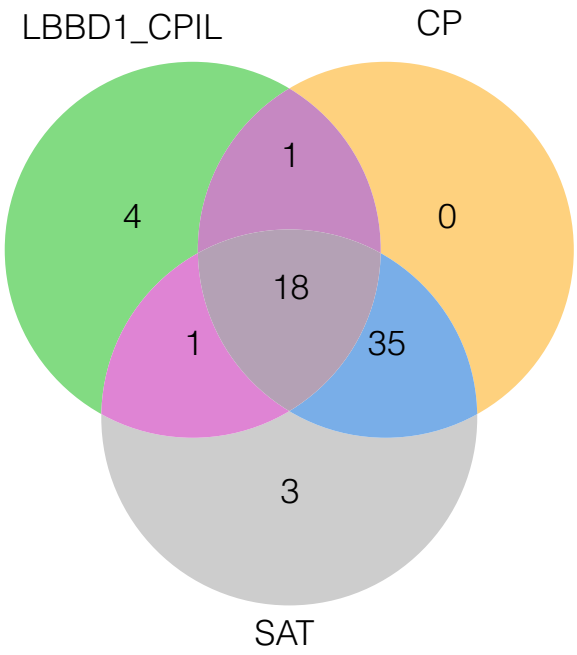


Figure 2. Venn diagram of instances proven optimal by LBBD1_CPIL (Xiong and Liu, 2026), CP (Meng et al., 2020; Xiong and Liu, 2026), and SAT across the two- and three-factory Brandimarte and *rdata* configurations.

Figure 2 complements the configuration-level comparison above

by showing how the optimality certificates of SAT, CP, and LBBD1_CPIL overlap at the instance level across the Brandimarte and *rdata* benchmarks. The comparison covers 66 instances from the two- and three-factory configurations, excluding Fattahi because corresponding CP and LBBD1_CPIL results are unavailable.[2] [[2] For this figure, the CP set is the union of the original optimality results reported by Meng et al. (2020) and the CP rerun reported by Xiong and Liu (2026), with duplicate instances counted only once. Instances not evaluated by LBBD1_CPIL remain in the 66-instance benchmark set but are not counted in the LBBD1_CPIL set.] SAT extends the optimality coverage beyond that established by CP, proving four instances that are not certified by CP, including three SAT-only instances and one instance also certified by LBBD1_CPIL. At the same time, the overlap between the two approaches is substantial, with 53 of the 54 instances proven optimal by CP also being certified by SAT. Thus, SAT adds new optimality certificates while covering nearly the entire set previously established by CP.

LBBD1_CPIL provides further complementary optimality certificates. Five of its reported certificates lie outside the SAT set, including four unique to LBBD1_CPIL and one shared with CP. Considering the union of the three exact approaches, optimality is established for 62 of the 66 Brandimarte and *rdata* instances, leaving only four without a certificate among the available results. Together with the 10 Fattahi instances already proven optimal, 72 of the 76 benchmark instances considered in this study now have established optimal makespans.

Table 4. Benchmark instances for which SAT improves the previously best-known makespan or proves optimality.

Instance	#Factories	BKV	SAT		Improvement
			Makespan	t (s)	
MK10	2	130	**125***	722.8	5
MK08	3	184	**184***	84.0	0
la15	3	388	**387***	127.9	1

- BKV denotes the best-known makespan previously reported in the literature.
- An asterisk (*) indicates that the corresponding makespan is proven optimal.

SAT also improves previously reported benchmark results, as summarized in Table 4. For *MK10* with two factories, the previously best-known makespan of 130 is reduced to 125, and the improved value is proven optimal. For *la15* with three factories, the best-known makespan is reduced from 388 to 387 and is

likewise certified optimal. In addition, SAT proves that the previously reported makespan of 184 for *MK08* with three factories is optimal. These results demonstrate that SAT can not only match existing benchmark results but also advance the state of the art by improving known bounds and establishing new optimality certificates.

The additional computational budget mainly affects the difficult two-factory cases. On Brandimarte, increasing the time limit from 600 s to 1800 s yields one additional optimality proof and one additional best makespan, including the improvement and certification of *MK10*. For the two-factory *rdata* configuration, the numbers of best and proven-optimal solutions remain unchanged, but the average deviation decreases from 0.39% to 0.24%, indicating that the longer runs improve incumbent solutions on some unresolved instances. Thus, the additional computational budget primarily strengthens solution quality and optimality certification on the remaining difficult cases rather than changing the overall performance pattern.

5. Conclusion

This paper has presented an exact SAT-based approach for the Distributed Flexible Job Shop Scheduling Problem (DFJSP) as a complementary alternative to existing exact methods. The proposed formulation jointly captures factory assignment, machine

selection, operation timing, precedence relations, and conditional machine-capacity conflicts, while exploiting problem structure through temporal-domain reduction, propagation, symmetry breaking, and incremental SAT solving. The resulting framework enables both effective search for feasible schedules and rigorous certification of optimality, as demonstrated by the computational results on standard DFJSP benchmarks.

Computational results demonstrate the effectiveness of the proposed approach across 76 standard benchmark instances. Together with previously established results, the new SAT optimality certificates bring the total number of proven-optimal instances to 72 out of 76, including three newly certified instances. Moreover, SAT improves the previous best-known makespans for two of these instances: *MK10* with two factories, from 130 to 125, and *la15* with three factories, from 388 to 387, with both improved makespans proven optimal. For *MK08* with three factories, SAT also proves the incumbent best-known makespan of 184 to be optimal. Across the benchmark configurations, SAT achieves the best results on four of the five configurations and remains competitive with strong exact approaches, including CP and LBB-D1_CPIL, as well as existing metaheuristic methods.

Future work will investigate stronger propagation mechanisms and more compact resource-conflict encodings for larger instances. Integrating SAT with heuristic warm-start strategies and

tighter lower bounds is another promising direction. The framework can also be extended to more complex DFJSP variants, including sequencing flexibility, heterogeneous factories, and transportation times, and evaluated on additional benchmark instances.

References

Biere et al. (2020)

A. Biere, K. Fazekas, M. Fleury, and M. Heisinger
CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020.
In Proc. of SAT Competition 2020 – Solver and Benchmark Descriptions, T. Balyo, N. Froleyks, M. Heule, M. Iser, M. Järvisalo, and M. Suda (Eds.), Department of Computer Science Report Series B, Vol. **B-2020-1**, pp. 51–53.
Cited by: §4.2.

Biere et al. (2009)

A. Biere, H. van Maaren, and T. Walsh
Handbook of satisfiability.

SAGE Publications Limited.
Cited by: §1.

Brandimarte (1993)

P. Brandimarte
Routing and scheduling in a flexible job shop by tabu search.
Annals of Operations Research **41** (3), pp. 157–183.
External Links: Document (https://dx.doi.org/10.1007/BF02023073)
Cited by: 2nd item, §1, §4.1, Table 3.

Brucker and Schlie (1990)

P. Brucker and R. Schlie

Job-shop scheduling with multi-purpose machines.
Computing **45** (4), pp. 369–375.
Cited by: §1.

Chang and Liu (2017)

H. C. Chang and T. K. Liu
Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms.
Journal of Intelligent Manufacturing **28**, pp. 1973–1986.

External Links: Document (https://dx.doi.org/10.1007/s10845-015-1084-y)
Cited by: §1, §2.2.

Coelho and Vanhoucke (2025)

J. Coelho and M. Vanhoucke

Comparing and extending satisfiability solution methods for the resource-constrained project scheduling problem. *Computers & Operations Research*, pp. 107308.
Cited by: §1.

Dauzère-Pérès et al. (2024)

S. Dauzère-Pérès, J. Ding, L. Shen, and K. Tamssaouet
The flexible job shop scheduling problem: a review. *European Journal of Operational Research* **314** (2), pp. 409–432.
External Links: Document (https://dx.doi.org/10.1016/j.ejor.2023.05.017)
Cited by: §1.

De Giovanni and Pezzella (2010)

L. De Giovanni and F. Pezzella

An improved genetic algorithm for the distributed and flexible job-shop scheduling problem. *European Journal of Operational Research* **200** (2), pp. 395–408.
External Links: Document (https://dx.doi.org/10.1016/j.ejor.2009.01.008)
Cited by: §1, §1, §2.2.

Fattahi et al. (2007)

P. Fattahi, M. Saidi Mehrabad, and F. Jolai
Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *Journal of Intelligent Manufacturing* **18** (3), pp. 331–342.
External Links: Document (https://dx.doi.org/10.1007/s10845-007-0026-8)
Cited by: §1, §4.1, Table 3.

Graham (1966)

R. L. Graham
Bounds for certain multiprocessing anomalies. *Bell system technical journal* **45** (9), pp. 1563–1581.

Cited by: §1.

Huang and Zhou (2018)

H. Huang and S. Zhou
An efficient sat algorithm for complex job-shop scheduling.
In 2018 8th International Conference on Manufacturing Science and Engineering (ICMSE 2018),
pp. 683–688.
Cited by: §1, §2.2.

Hurink et al. (1994)

J. Hurink, B. Jurisch, and M. Thole
Tabu search for the job-shop scheduling problem with multi-purpose machines.
OR Spectrum **15** (4), pp. 205–215.
External Links: Document (https://dx.doi.org/10.1007/BF01719451)
Cited by: 2nd item, §1, §4.1, Table 3.

Ignatiev et al. (2024)

A. Ignatiev, Z. L. Tan, and C. Karamanos
Towards universally accessible SAT technology.
In SAT,
pp. 4:1–4:11.

pp. [illegible]–[illegible].
External Links: Link (https://doi.org/10.4230/LIPIcs.SAT.2024.16), Document (https://dx.doi.org/10.4230/LIPICS.SAT.2024.16)
Cited by: §4.2.

Koshimura et al. (2010)
M. Koshimura, H. Nabeshima, H. Fujita, and R. Hasegawa
Solving open job-shop scheduling problems by sat encoding.
IEICE TRANSACTIONS on Information and Systems **93** (8), pp. 2316–2318.
Cited by: §1, §2.2.

Lu et al. (2018)
P. H. Lu, M. C. Wu, H. Tan, Y. H. Peng, and C. F. Chen
A genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems.
Journal of Intelligent Manufacturing **29**, pp. 19–34.
External Links: Document (https://dx.doi.org/10.1007/s10845-015-1083-z)

Cited by: §1, §2.2.

Marzouki et al. (2018)

B. Marzouki, O. B. Driss, and K. Ghedira
Solving distributed and flexible job shop scheduling problem using a chemical reaction optimization metaheuristic.
Procedia Computer Science **126**, pp. 1424–1433.
External Links: Document (https://dx.doi.org/10.1016/j.procs.2018.08.114)
Cited by: §1, §2.2.

Mastrolilli and Gambardella (2000)

M. Mastrolilli and L. M. Gambardella
Effective neighborhood functions for the flexible job shop problem.
Journal of Scheduling - SCHEDULING **3**, pp. .
External Links: Document (https://dx.doi.org/10.1002/%28SICI%291099-1425%28200001/02%293%3A1%3C3%3A%3AAID-JOS32%3E3.0.CO%3B2-Y)
Cited by: §1.

Meng et al. (2020) L. Meng, C. Zhang, Y. Ren, B. Zhang, and C. Lv
Mixed-integer linear programming and constraint programming formulations for solving distributed flexible job shop scheduling problem.
Computers & Industrial Engineering **142**, pp. 106347.
External Links: Document (https://dx.doi.org/10.1016/j.cie.2020.106347)
Cited by: §1, §1, §2.2, Figure 2, §4.1, §4.2, §4.3, Table 3, Table 3, footnote 2.

Xie et al. (2023) J. Xie, X. Li, L. Gao, and L. Gui
A hybrid genetic tabu search algorithm for distributed flexible job shop scheduling problems.
Journal of Manufacturing Systems **71**, pp. 82–94.
External Links: Document (https://dx.doi.org/10.1016/j.jmsy.2023.09.002)
Cited by: §1, §2.2, §4.1, §4.2, §4.2, Table 3.

Xiong and Liu (2026) F. Xiong and H. Liu Logic-based benders decomposition methods for the distributed flexible job shop scheduling problem. *European Journal of Operational Research* **329** (3), pp. 778–797.
External Links: Document (https://dx.doi.org/10.1016/j.ejor.2025.08.039)
Cited by: §1, §2.2, Figure 2, item *, §4.2, §4.2, §4.3, Table 3, Table 3, footnote 2.

Experimental support, please view the build logs for errors. Generated by LaTeXML (https://math.nist.gov/~BMiller/LaTeXML/).

Instructions for reporting errors

We are continuing to improve HTML versions of papers, and your feedback helps enhance accessibility and mobile support. To report errors in the HTML that will help us improve conversion and rendering, choose any of the methods listed below:

- Click the "Report Issue" (🐞) button, located in the page header.

Tip: You can select the relevant text first, to include it in your report.

Our team has already identified the following issues (https://github.com/arXiv/html_feedback/issues). We appreciate your time reviewing and reporting rendering errors we may not have found yet. Your efforts will help us improve the HTML versions for all readers, because disability should not be a barrier to accessing research. Thank you for your continued support in championing open access for all.

Have a free development cycle? Help support accessibility at arXiv! Our collaborators at LaTeXML maintain a list of packages that need conversion (https://github.com/brucemiller/LaTeXML/wiki/Porting-LaTeX-packages-for-LaTeXML), and welcome developer contributions (https://github.com/brucemiller/LaTeXML/issues).